



INTAKE 46

Bash Scripting

>_ User & Group Management Tool | ITI Alexandria

Track: System Administration

Supervised By

>_ Eng. Romany Nageh

Prepared By

>_ Ahmed Gamal

Contents

1	Introduction	2
1.1	What is Whiptail?	2
1.2	How to Run the Script	2
2	Script Structure Overview	2
3	Helper Functions	2
3.1	msg() – Display a Message	2
3.2	confirm() – Ask a Yes/No Question	3
4	User Management Functions	3
4.1	add_user() – Create a New User	3
4.2	modify_user() – Rename a User	4
4.3	delete_user() – Remove a User	4
4.4	list_users() – Show All Regular Users	5
5	Group Management Functions	5
5.1	add_group() – Create a New Group	5
5.2	modify_group() – Add a User to a Group	6
5.3	delete_group() – Remove a Group	6
5.4	list_groups() – Show All Groups	7
6	Account Control Functions	7
6.1	disable_user() – Lock an Account	7
6.2	enable_user() – Unlock an Account	8
6.3	change_password() – Set a New Password	8
7	Main Menu Loop	9
8	Testing Scenarios	10
8.1	Scenario 1 – Add a New User	10
8.2	Scenario 2 – Add a New Group	11
8.3	Scenario 3 – Add the User to the Group	12
8.4	Scenario 4 – List Users and Groups	13
8.5	Scenario 5 – Disable (Lock) the User	14
8.6	Scenario 6 – Enable (Unlock) the User	15
8.7	Scenario 7 – Change the User’s Password	16
8.8	Scenario 8 – Delete the User	17
8.9	Scenario 9 – Delete the Group	19
9	Command Reference Table	20
10	About	21

1 Introduction

This document describes a Bash shell script called the **User & Group Management Tool**. The script provides an interactive menu inside the terminal (called a TUI – Text User Interface) built using **whiptail**. It allows a system administrator to manage Linux users and groups easily without needing to memorize every command.

1.1 What is Whiptail?

whiptail is a standard Linux utility that draws dialog boxes directly inside the terminal window. It can display menus, input boxes, password fields, yes/no questions, and message boxes – all without a graphical desktop. It ships by default on most Linux distributions including RedHat and Debian.

1.2 How to Run the Script

Save the script as `usermgmt.sh`, make it executable, then run it with root privileges:

```
1 chmod +x usermgmt.sh
2 sudo ./usermgmt.sh
```

Important: All user and group management commands on Linux require root privileges. Always run this script with **sudo** or as the **root** user.

2 Script Structure Overview

The script is organized into five clear sections:

1. **Helper functions** – small shared functions used everywhere in the script.
2. **User functions** – add, modify, delete, and list system users.
3. **Group functions** – add, modify, delete, and list system groups.
4. **Account control** – lock/unlock accounts and change passwords.
5. **Main menu loop** – the interactive menu that connects everything.

3 Helper Functions

3.1 msg() – Display a Message

This small function is used throughout the script to show a result message to the user. The button is labeled **Back** so the user knows they are returning to the main menu.

```
1 # -----
2 # msg() : Show an info message box with a "Back" button.
3 # Usage : msg "Your message here"
4 # -----
5 msg() {
6     whiptail --title "Info" --ok-button "Back" --msgbox "$1" 12 55
7 }
```

Explanation:

- \$1 is the message text passed when calling the function.
- 12 55 sets the dialog box height (12 lines) and width (55 characters).
- -ok-button "Back" renames the default OK button to Back.

3.2 confirm() – Ask a Yes/No Question

This function is called before any destructive action (like deleting a user or group). It returns 0 if the user pressed Yes, and 1 if they pressed No.

```

1 # -----
2 # confirm() : Show a Yes/No dialog.
3 # Returns   : 0 = Yes, 1 = No
4 # Usage     : confirm "Are you sure?"
5 # -----
6 confirm() {
7     whiptail --title "Confirm" --yesno "$1" 8 50
8 }

```

4 User Management Functions

4.1 add_user() – Create a New User

This function prompts for a username and password, creates the user account with a home directory, and sets their password.

```

1 add_user() {
2     # Step 1: Ask for the new username
3     USERNAME=$(whiptail --title "Add User" \
4         --inputbox "Enter new username:" 8 50 3>&1 1>&2 2>&3)
5     [ -z "$USERNAME" ] && return      # Empty input = go back to menu
6
7     # Step 2: Ask for the password (hidden input)
8     PASSWORD=$(whiptail --title "Add User" \
9         --passwordbox "Enter password for '$USERNAME':" \
10        8 50 3>&1 1>&2 2>&3)
11    [ -z "$PASSWORD" ] && return
12
13    # Step 3: Create the user with a home directory (-m flag)
14    useradd -m "$USERNAME" 2>/dev/null
15    if [ $? -ne 0 ]; then
16        msg "ERROR: Could not create user. Maybe it already exists?"
17        return
18    fi
19
20    # Step 4: Set the password using chpasswd
21    echo "$USERNAME:$PASSWORD" | chpasswd 2>/dev/null
22    msg "User '$USERNAME' created successfully."
23 }

```

Key commands explained:

- **useradd -m** – creates the user and automatically creates their home directory at `/home/username`.
- **chpasswd** – reads `username:password` from standard input and sets the password without interaction.

- `3>&1 1>&2 2>&3` – a redirect trick that lets whiptail capture what the user typed and return it to the variable.
- `[ -z "$VAR" ] && return` – if the user left the input empty or pressed Cancel, go back.

4.2 modify_user() – Rename a User

Renames an existing user account to a new login name.

```

1 modify_user() {
2     USERNAME=$(whiptail --title "Modify User" \
3         --inputbox "Enter username to modify:" 8 50 3>&1 1>&2 2>&3)
4     [ -z "$USERNAME" ] && return
5
6     # Verify the user actually exists before proceeding
7     if ! id "$USERNAME" &>/dev/null; then
8         msg "ERROR: User '$USERNAME' does not exist."
9         return
10    fi
11
12    NEWNAME=$(whiptail --title "Modify User" \
13        --inputbox "Enter new username (leave blank to cancel):" \
14        9 55 3>&1 1>&2 2>&3)
15    [ -z "$NEWNAME" ] && return
16
17    # Rename the login name with usermod -l
18    usermod -l "$NEWNAME" "$USERNAME" 2>/dev/null
19    if [ $? -eq 0 ]; then
20        msg "User '$USERNAME' renamed to '$NEWNAME'."
21    else
22        msg "ERROR: Could not rename. Make sure the user is not
23            logged in."
24    fi
25 }
```

Key command: `usermod -l newname oldname` changes only the login name. The home directory is not renamed automatically.

4.3 delete_user() – Remove a User

Permanently deletes a user account and their home directory. Always asks for confirmation first.

```

1 delete_user() {
2     USERNAME=$(whiptail --title "Delete User" \
3         --inputbox "Enter username to delete:" 8 50 3>&1 1>&2 2>&3)
4     [ -z "$USERNAME" ] && return
5
6     if ! id "$USERNAME" &>/dev/null; then
7         msg "ERROR: User '$USERNAME' does not exist."
8         return
9     fi
10
11    # Ask for confirmation before the irreversible delete
12    confirm "Delete '$USERNAME'? This also removes their home
13        directory!" \
14        || return
15
16    # -r flag removes the home directory as well
```

```

16     userdel -r "$USERNAME" 2>/dev/null
17     if [ $? -eq 0 ]; then
18         msg "User '$USERNAME' deleted successfully."
19     else
20         msg "ERROR: Could not delete user '$USERNAME'."
21     fi
22 }

```

Key command: `userdel -r` deletes the user AND removes their entire `/home/username` directory and mail spool.

4.4 `list_users()` – Show All Regular Users

Reads the `/etc/passwd` file and displays all regular users. System accounts have UIDs below 500 and are automatically skipped.

```

1  list_users() {
2      # Use awk to read /etc/passwd (colon-separated fields)
3      # Field $1 = username, $3 = UID, $6 = home directory
4      # Show only users with UID >= 500 (real human users)
5      USERS=$(awk -F: '
6          $3 >= 500 && $1 != "nobody" && $1 != "nfsnobody" {
7              printf "%-15s UID:%-6s %s\n", $1, $3, $6
8          }
9      ' /etc/passwd)
10
11     if [ -z "$USERS" ]; then
12         USERS="No regular users found."
13     fi
14
15     # scrolltext allows scrolling when there are many users
16     whiptail --title "List Users" --scrolltext \
17         --msgbox "Username          UID      Home Dir\n\
18         -----\n$USERS" 20 60
19 }

```

Key concept: The file `/etc/passwd` stores one user per line, with fields separated by colons. `awk -F:` splits each line on the colon character so we can access individual fields.

5 Group Management Functions

5.1 `add_group()` – Create a New Group

```

1  add_group() {
2      GROUPNAME=$(whiptail --title "Add Group" \
3          --inputbox "Enter new group name:" 8 50 3>&1 1>&2 2>&3)
4      [ -z "$GROUPNAME" ] && return
5
6      groupadd "$GROUPNAME" 2>/dev/null
7      if [ $? -eq 0 ]; then
8          msg "Group '$GROUPNAME' created successfully."
9      else
10         msg "ERROR: Could not create group. Maybe it already exists?"
11     fi
12 }

```

Key command: `groupadd groupname` creates a new entry in `/etc/group` with the next available Group ID (GID).

5.2 `modify_group()` – Add a User to a Group

```

1 modify_group() {
2     GROUPNAME=$(whiptail --title "Modify Group" \
3         --inputbox "Enter group name to modify:" 8 50 3>&1 1>&2
4         2>&3)
5     [ -z "$GROUPNAME" ] && return
6
7     # Check the group exists first
8     if ! getent group "$GROUPNAME" &>/dev/null; then
9         msg "ERROR: Group '$GROUPNAME' does not exist."
10        return
11    fi
12
13    USERNAME=$(whiptail --title "Modify Group" \
14        --inputbox "Enter username to add to '$GROUPNAME':" \
15        8 50 3>&1 1>&2 2>&3)
16    [ -z "$USERNAME" ] && return
17
18    if ! id "$USERNAME" &>/dev/null; then
19        msg "ERROR: User '$USERNAME' does not exist."
20        return
21    fi
22
23    # -aG : append (-a) user to this supplementary group (-G)
24    # The -a flag is critical -- without it, the user is REMOVED
25    # from all their current groups and only added to this one.
26    usermod -aG "$GROUPNAME" "$USERNAME" 2>/dev/null
27    if [ $? -eq 0 ]; then
28        msg "User '$USERNAME' added to group '$GROUPNAME'."
29    else
30        msg "ERROR: Could not modify group '$GROUPNAME'."
31    fi
32 }
```

Critical note about -aG: Always use `-a` (append) together with `-G`. If you use only `-G`, the user is removed from all their existing groups and placed only in the new one – which can cause permission problems.

5.3 `delete_group()` – Remove a Group

```

1 delete_group() {
2     GROUPNAME=$(whiptail --title "Delete Group" \
3         --inputbox "Enter group name to delete:" 8 50 3>&1 1>&2
4         2>&3)
5     [ -z "$GROUPNAME" ] && return
6
7     if ! getent group "$GROUPNAME" &>/dev/null; then
8         msg "ERROR: Group '$GROUPNAME' does not exist."
9         return
10    fi
11
12    confirm "Delete group '$GROUPNAME'?" || return
```

```

13     groupdel "$GROUPNAME" 2>/dev/null
14     if [ $? -eq 0 ]; then
15         msg "Group '$GROUPNAME' deleted."
16     else
17         msg "ERROR: Cannot delete. It may be a user's primary group."
18     fi
19 }

```

Note: A group cannot be deleted if it is the *primary group* of any user. You must delete or reassign those users first.

5.4 list_groups() – Show All Groups

Reads /etc/group line by line and displays each group with its GID and list of members.

```

1 list_groups() {
2     OUTPUT=""
3     # /etc/group format:  groupname : password : GID : member1,
4     #                      member2
5     while IFS=: read -r GNAME _ GID MEMBERS; do
6         # If members field is empty, show placeholder text
7         [ -z "$MEMBERS" ] && MEMBERS="(no members)"
8         OUTPUT="${OUTPUT}$(printf '%-20s GID:%-6s %s\n' \
9             "$GNAME" "$GID" "$MEMBERS")"$'\n'
10    done < /etc/group
11
12    whiptail --title "List Groups" --scrolltext \
13        --msgbox "Group Name          GID      Members\n\
14        -----\n${OUTPUT}" 22 65
15 }

```

6 Account Control Functions

6.1 disable_user() – Lock an Account

Locking a user prevents them from logging in without deleting their account. The lock is achieved by adding an exclamation mark (!) in front of the encrypted password in /etc/shadow, making the hash invalid.

```

1 disable_user() {
2     USERNAME=$(whiptail --title "Disable User" \
3         --inputbox "Enter username to lock:" 8 50 3>&1 1>&2 2>&3)
4     [ -z "$USERNAME" ] && return
5
6     if ! id "$USERNAME" &>/dev/null; then
7         msg "ERROR: User '$USERNAME' does not exist."
8         return
9     fi
10
11     # -L flag = Lock (adds ! before the password hash in /etc/shadow)
12     usermod -L "$USERNAME" 2>/dev/null
13
14     if [ $? -eq 0 ]; then
15         # Confirm the lock by reading /etc/shadow directly

```

```

16     # Field 2 is the password hash -- it should now start with !
17     SHADOW_ENTRY=$(grep "^$USERNAME:" /etc/shadow | cut -d: -f2)
18     if [[ "$SHADOW_ENTRY" == !* ]]; then
19         msg "User '$USERNAME' is now LOCKED.\nVerify: passwd -S
           $USERNAME"
20     fi
21 else
22     msg "ERROR: Could not lock user '$USERNAME'."
23 fi
24 }

```

How to verify from the terminal:

```

1 passwd -S testuser
2 # Output example:
3 # testuser L 2026-04-16 0 99999 7 -1 (Password locked.)
4 #
5 #           L = Locked      P = Active (password set)

```

6.2 enable_user() – Unlock an Account

```

1 enable_user() {
2     USERNAME=$(whiptail --title "Enable User" \
3         --inputbox "Enter username to unlock:" 8 50 3>&1 1>&2 2>&3)
4     [ -z "$USERNAME" ] && return
5
6     if ! id "$USERNAME" &>/dev/null; then
7         msg "ERROR: User '$USERNAME' does not exist."
8         return
9     fi
10
11     # -U flag = Unlock (removes the ! from the password hash)
12     usermod -U "$USERNAME" 2>/dev/null
13
14     if [ $? -eq 0 ]; then
15         SHADOW_ENTRY=$(grep "^$USERNAME:" /etc/shadow | cut -d: -f2)
16         if [[ "$SHADOW_ENTRY" != !* ]]; then
17             msg "User '$USERNAME' is now UNLOCKED.\nVerify: passwd -
              S $USERNAME"
18         fi
19     else
20         msg "ERROR: Could not unlock user '$USERNAME'."
21     fi
22 }

```

6.3 change_password() – Set a New Password

```

1 change_password() {
2     USERNAME=$(whiptail --title "Change Password" \
3         --inputbox "Enter username:" 8 50 3>&1 1>&2 2>&3)
4     [ -z "$USERNAME" ] && return
5
6     if ! id "$USERNAME" &>/dev/null; then
7         msg "ERROR: User '$USERNAME' does not exist."
8         return
9     fi

```

```

10
11     PASSWORD=$(whiptail --title "Change Password" \
12         --passwordbox "Enter new password for '$USERNAME':" \
13         8 50 3>&1 1>&2 2>&3)
14     [ -z "$PASSWORD" ] && return
15
16     # chpasswd reads "username:password" from stdin and updates /etc
17     # /shadow
18     echo "$USERNAME:$PASSWORD" | chpasswd 2>/dev/null
19     if [ $? -eq 0 ]; then
20         msg "Password for '$USERNAME' changed successfully."
21     else
22         msg "ERROR: Could not change password."
23     fi
24 }

```

7 Main Menu Loop

The main menu runs in an infinite `while true` loop. Each time the user selects an option, the matching function is called, it runs, returns, and the menu appears again. The loop only exits when the user presses the **Exit** button or **Esc**.

```

1  while true; do
2      # Show the menu and store the chosen option number in CHOICE
3      CHOICE=$(whiptail --title "User & Group Management" \
4          --cancel-button "Exit" \
5          --ok-button "Select" \
6          --menu "Use arrow keys, press Enter to select:" 22 65 12 \
7          "1" "Add User" - Add a user to the system" \
8          "2" "Modify User" - Modify an existing user" \
9          "3" "Delete User" - Delete an existing user" \
10         "4" "List Users" - List all users on the system" \
11         "5" "Add Group" - Add a group to the system" \
12         "6" "Modify Group" - Add a user to a group" \
13         "7" "Delete Group" - Delete an existing group" \
14         "8" "List Groups" - List all groups" \
15         "9" "Disable User" - Lock the user account" \
16         "10" "Enable User" - Unlock the user account" \
17         "11" "Change Password" - Change password of a user" \
18         "12" "About" - About this program" \
19         3>&1 1>&2 2>&3)
20
21     # $? holds the exit code of the last command.
22     # whiptail returns non-zero when user presses Exit or Esc.
23     [ $? -ne 0 ] && break
24
25     # Call the function that matches the selected option
26     case $CHOICE in
27         1) add_user ;; 2) modify_user ;;
28         3) delete_user ;; 4) list_users ;;
29         5) add_group ;; 6) modify_group ;;
30         7) delete_group ;; 8) list_groups ;;
31         9) disable_user ;; 10) enable_user ;;
32         11) change_password ;; 12) about ;;
33     esac

```

```
34  
35 done  
36  
37 clear  
38 echo "Goodbye!"
```

8 Testing Scenarios

8.1 Scenario 1 – Add a New User

Goal: Create a user named `testuser` with password `redhat`.

» Steps to Perform

1. Run the script
2. Select option - "Add User"
3. Enter username: `testuser`, Enter password: `redhat`
4. Expected message: "User '`testuser`' created successfully."
5. Verify in terminal: `id testuser`

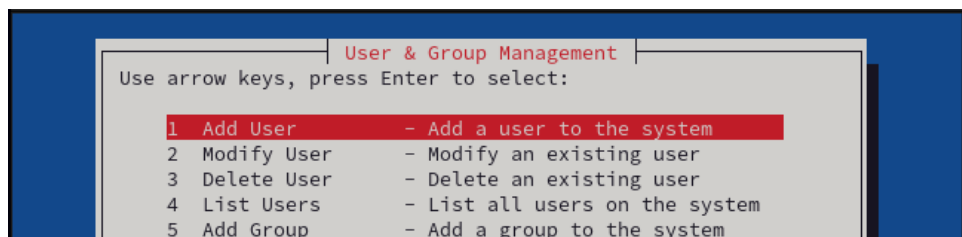


Figure 1: Main menu – option 1 highlighted

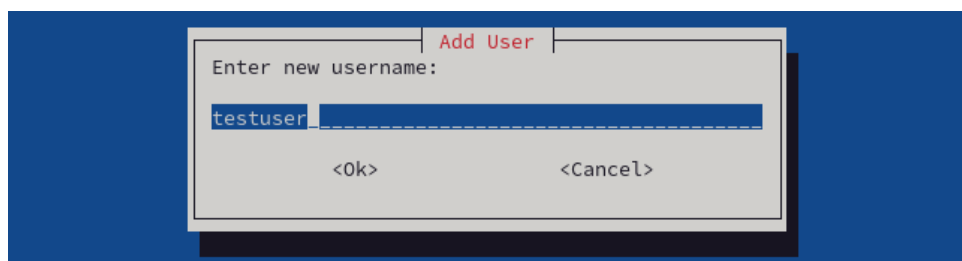


Figure 2: Add User input box – typing the username testuser

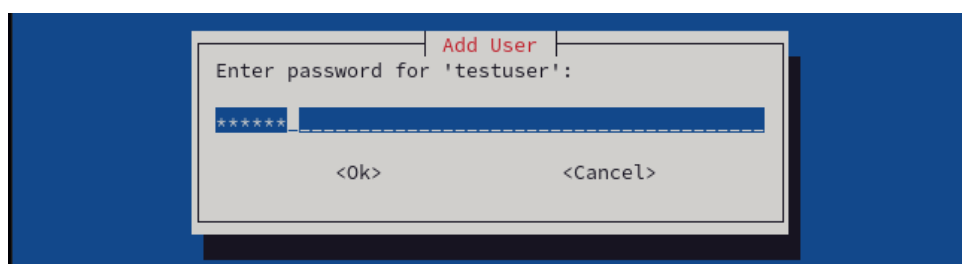


Figure 3: Password input box

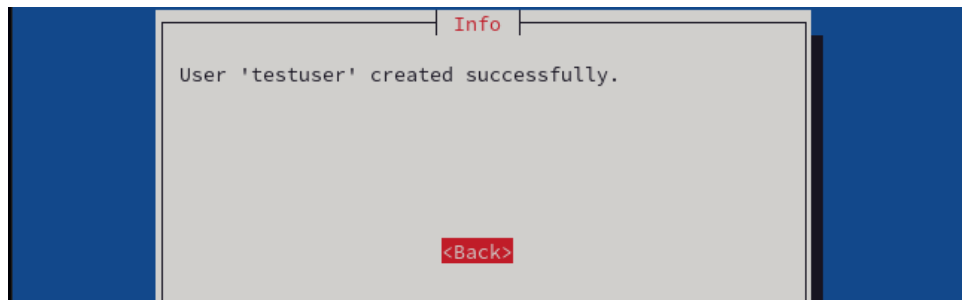


Figure 4: Success message dialog

```
[root@target bin]# id testuser
uid=1340(testuser) gid=1340(testuser) groups=1340(testuser)
[root@target bin]#
```

Figure 5: Terminal: id test user

8.2 Scenario 2 – Add a New Group

Goal: Create a group named sysadmin.

» Steps to Perform

1. Select option - "Add Group"
2. Enter group name: sysadmin
3. Expected message: "Group 'sysadmin' created successfully."
4. Verify in terminal: `grep sysadmin /etc/group`

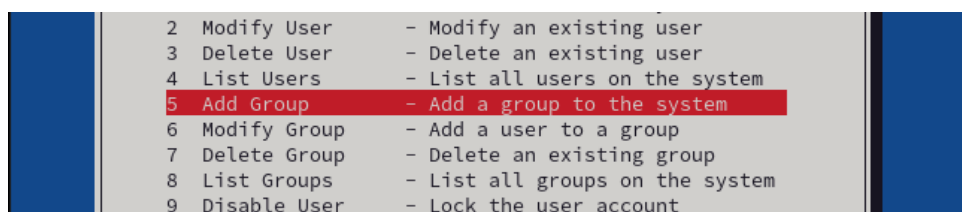


Figure 6: Choosing the option to add a group

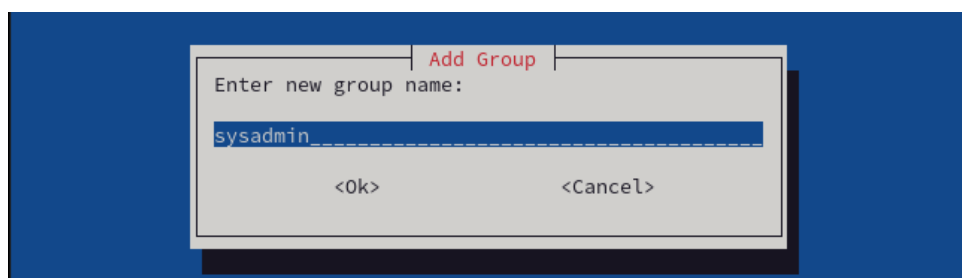


Figure 7: Add Group input box – typing sysadmin

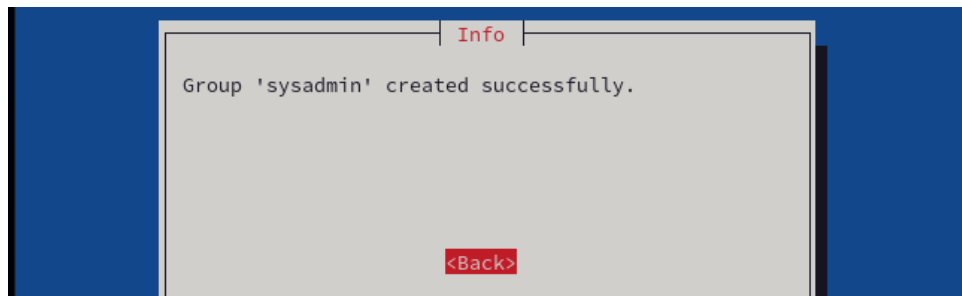


Figure 8: Success message after group creation

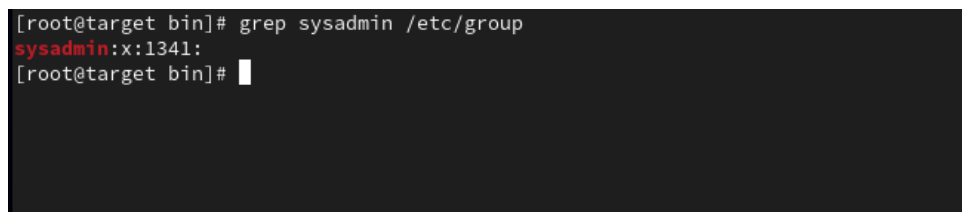


Figure 9: Terminal: grep sysadmin /etc/group output

8.3 Scenario 3 – Add the User to the Group

Goal: Add `testuser` into the `devteam` group.

» Steps to Perform

1. Select option - "Modify Group"
2. Enter group name: `sysadmin`
3. Enter username: `testuser`
4. Expected message: "User 'testuser' added to group 'sysadmin'."
5. Verify in terminal: `groups testuser grep devteam /etc/group`

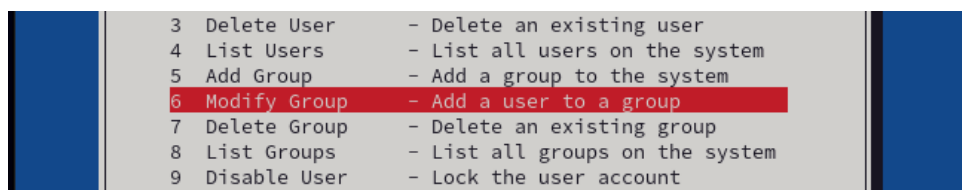


Figure 10: Choosing the option to modify a group

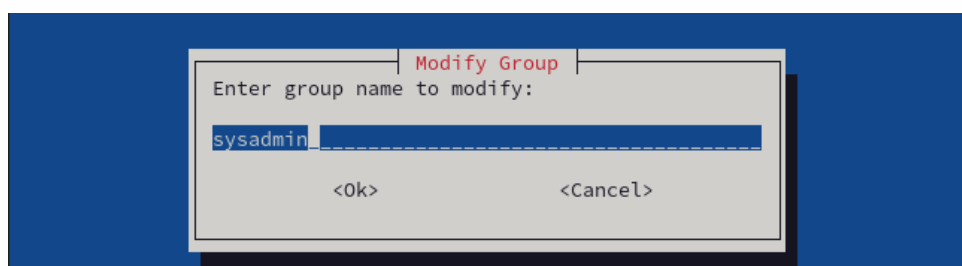


Figure 11: Modify Group – entering group name sysadmin

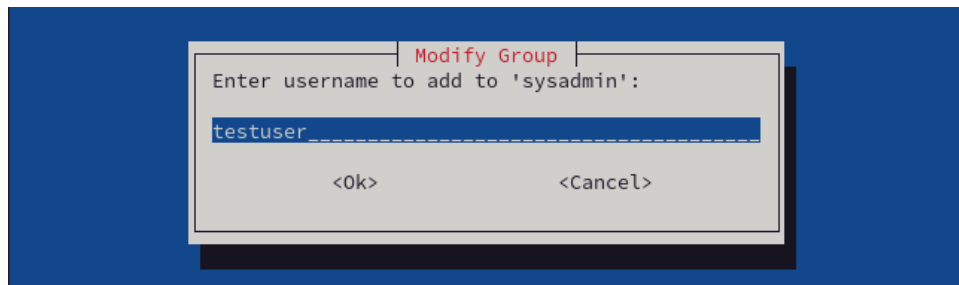


Figure 12: Modify Group – entering username testuser

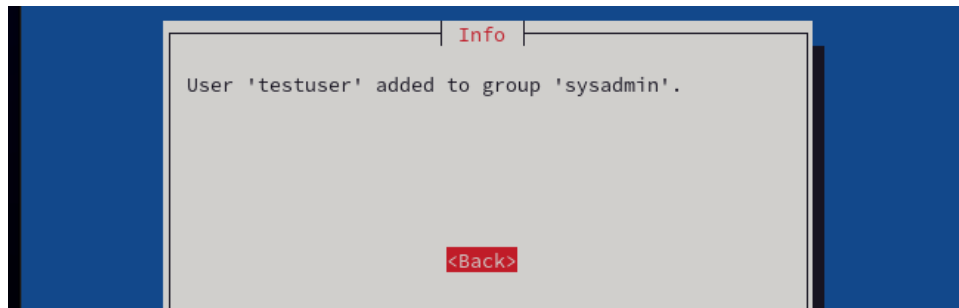


Figure 13: Success message – user added to group

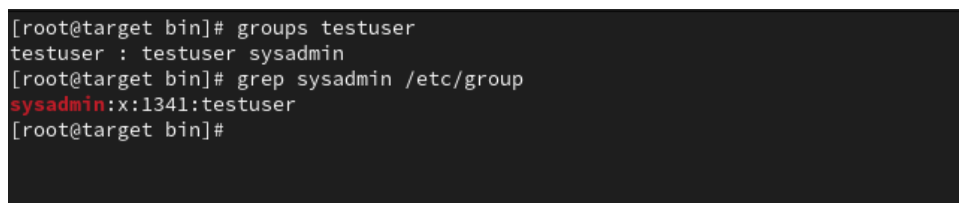


Figure 14: Terminal: groups testuser showing sysadmin membership

8.4 Scenario 4 – List Users and Groups

Goal: Verify testuser and sysadmin appear in the lists.

» Steps to Perform

1. Select option 4 - "List Users"
- > Confirm testuser is visible in the list
2. Press Back, then select option 8 - "List Groups"
- > Confirm sysadmin shows testuser as a member

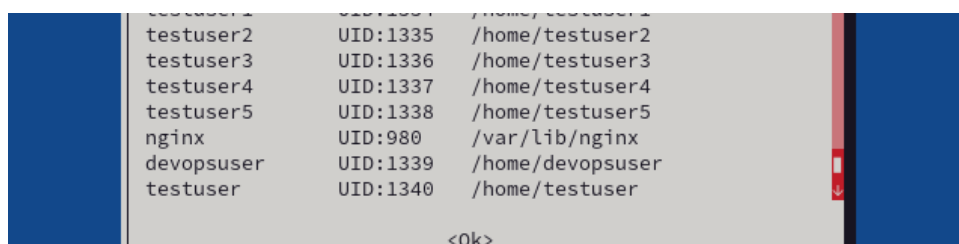


Figure 15: List Users screen – testuser visible



Figure 16: List Groups screen – sysadmin with testuser as member

8.5 Scenario 5 – Disable (Lock) the User

Goal: Lock testuser so they cannot log in.

» Steps to Perform

1. Select option - "Disable User"
2. Enter username: testuser
3. Expected message: "User 'testuser' is now LOCKED."
4. Verify in terminal: `passwd -S testuser`
Expected: testuser L ... (L = Locked)
5. Also check shadow: `grep -w "testuser" /etc/shadow`
Expected: password hash starts with !

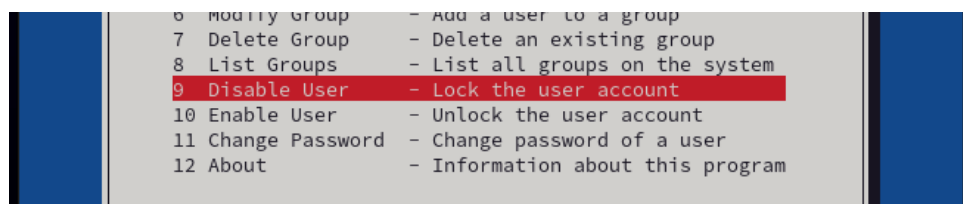


Figure 17: Choosing the option to disable a user

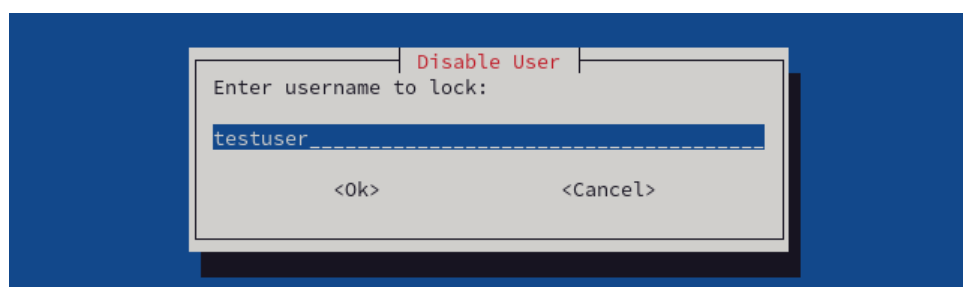


Figure 18: Disable User input box – typing testuser

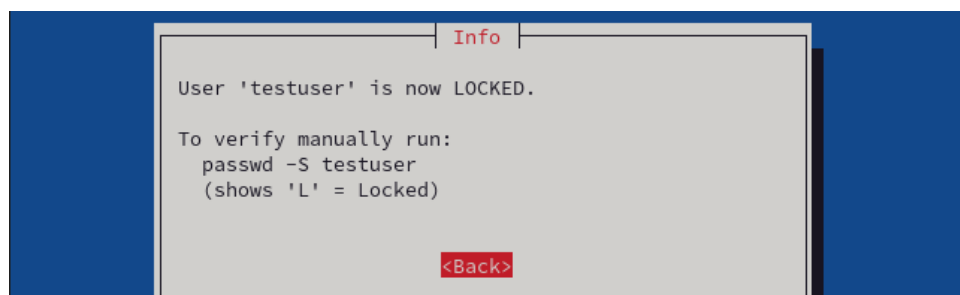


Figure 19: LOCKED confirmation message in TUI

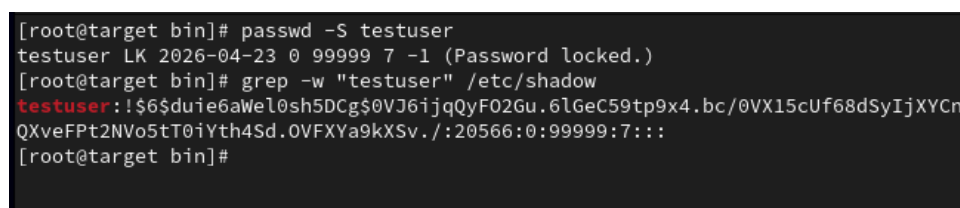


Figure 20: Terminal: passwd -S testuser showing L (Locked) and also grep testuser from /etc/shadow

8.6 Scenario 6 – Enable (Unlock) the User

Goal: Unlock testuser to restore access.

» Steps to Perform

1. Select option "Enable User"
 2. Enter username: testuser
 3. Expected message: "User 'testuser' is now UNLOCKED."
 4. Verify in terminal: passwd -S testuser
- Expected: testuser P ... (P = Password set / Active)

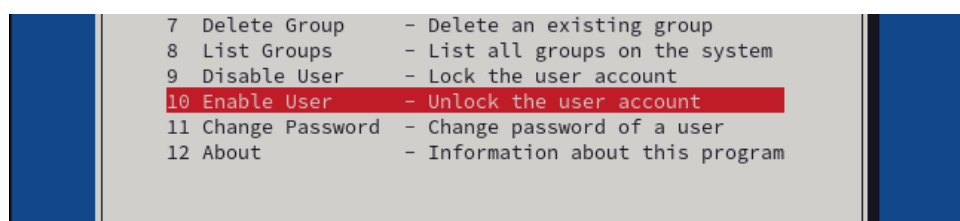


Figure 21: Choosing the option to enable a user

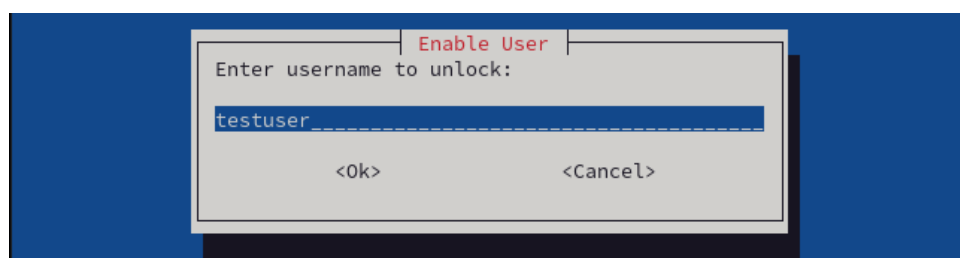


Figure 22: Enable User input box – typing testuser

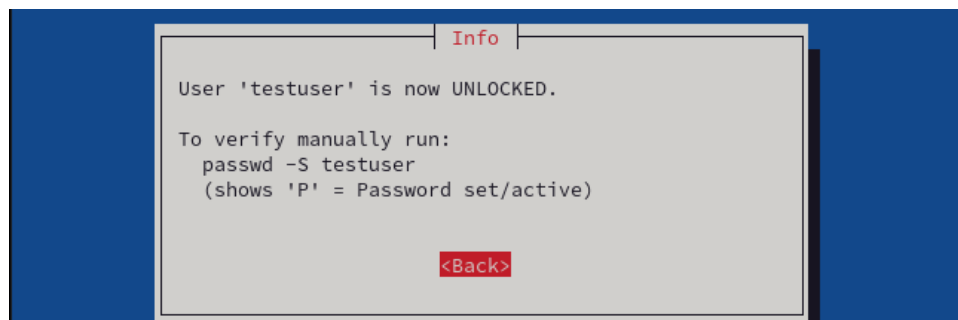


Figure 23: UNLOCKED confirmation message in TUI

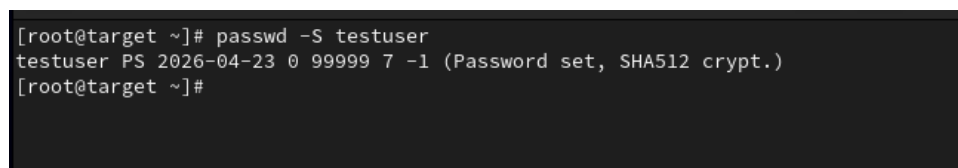


Figure 24: Terminal: passwd -S testuser showing P (Active)

8.7 Scenario 7 – Change the User's Password

Goal: Change testuser's password to redhat123.

» Steps to Perform

1. Select option 11 - "Change Password"
2. Enter username: testuser
3. Enter new password: redhat123
4. Expected message: "Password for 'testuser' changed successfully."
5. Verify: su - testuser (login with new password)

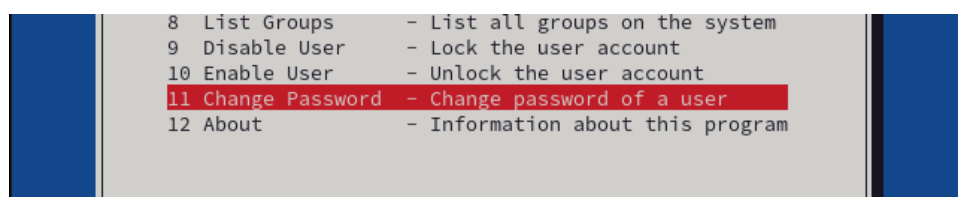


Figure 25: Choosing the option to Change Password

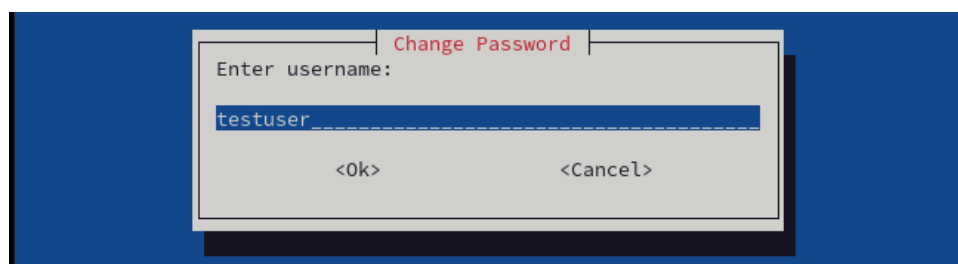


Figure 26: Change Password – entering username

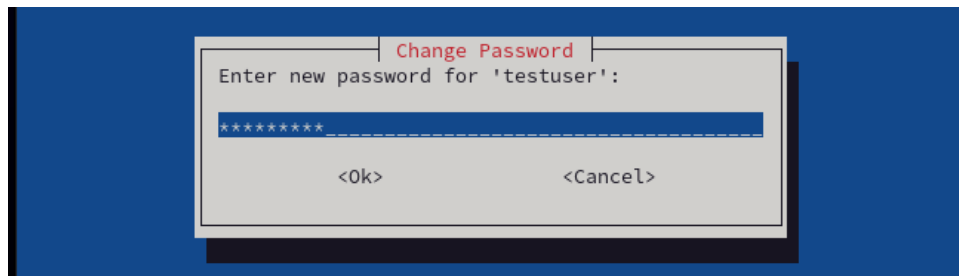


Figure 27: Change Password – entering new password

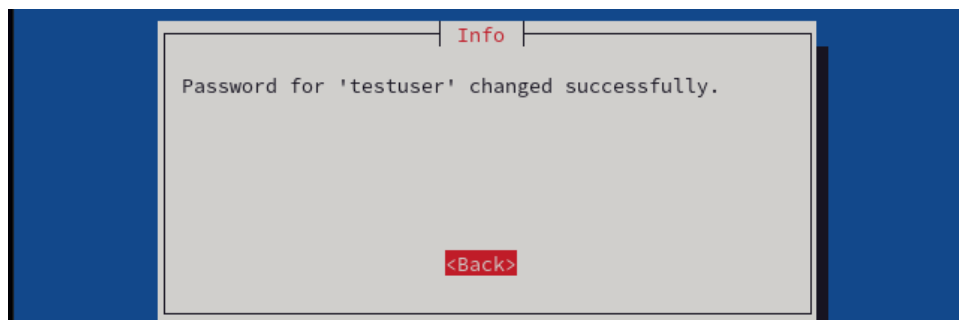


Figure 28: Success message after password change

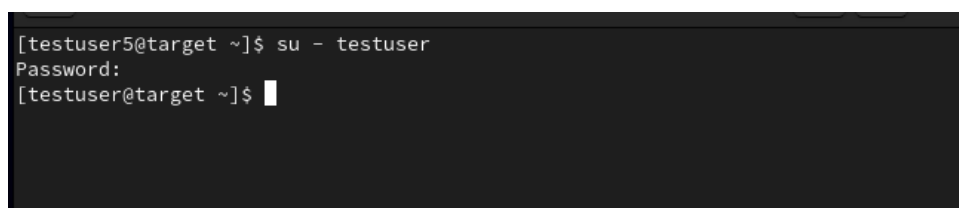


Figure 29: Terminal: su - testuser login with new password succeeding

8.8 Scenario 8 – Delete the User

Goal: Delete `testuser` and their home directory.

» Steps to Perform

1. Select option 3 - "Delete User"
2. Enter username: `testuser`
3. Confirm the Yes/No dialog -> select Yes
4. Expected message: "User 'testuser' deleted successfully."
5. Verify in terminal: `id testuser` (no such user)
`ls /home/testuser` (no such file)

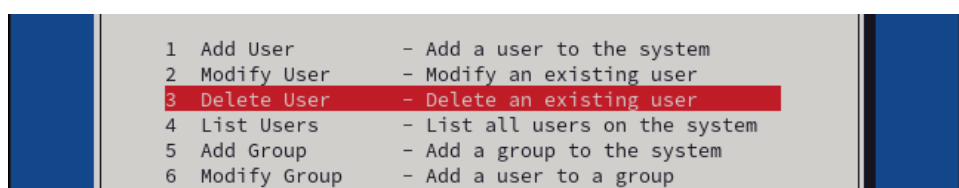


Figure 30: Choosing the option to Delete User

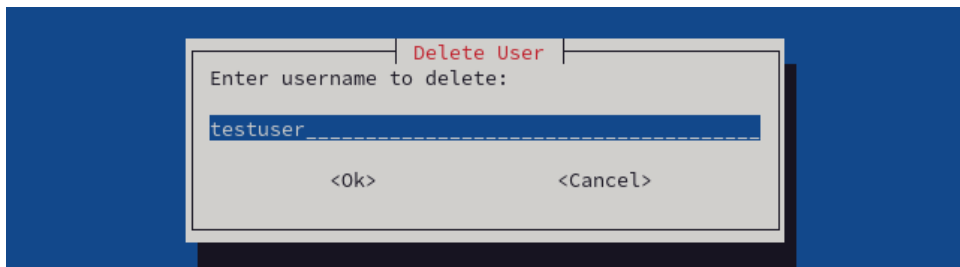


Figure 31: Add username of user you need to delete

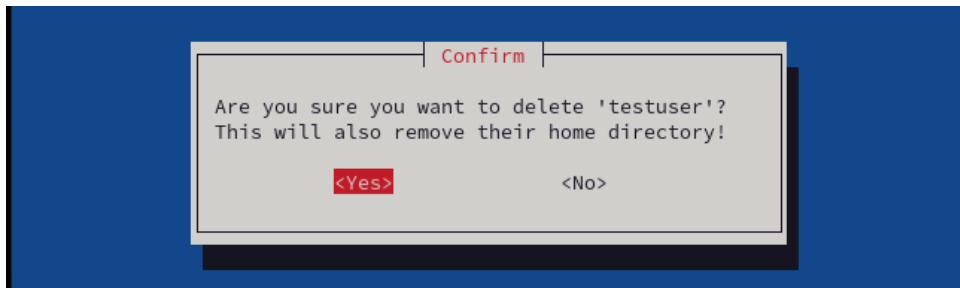


Figure 32: Delete User – Yes/No confirmation dialog to delete testuser with its home directory

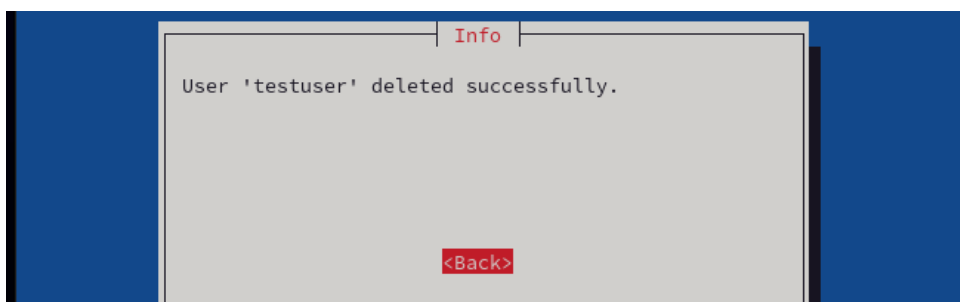


Figure 33: Success message after deletion

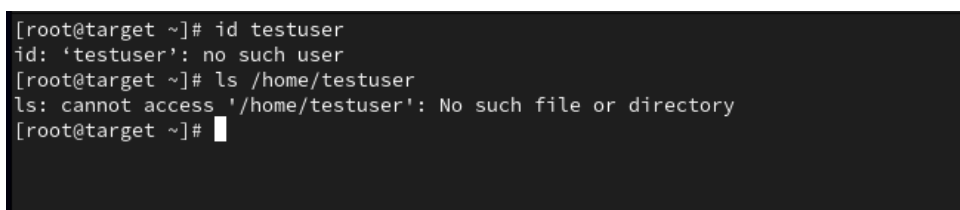


Figure 34: Terminal: id testuser – no such user and ls /home/testuser

8.9 Scenario 9 – Delete the Group

Goal: Delete the devteam group.

» Steps to Perform

1. Select option 7 - "Delete Group"
2. Enter group name: sysadmin
3. Confirm the Yes/No dialog -> select Yes
4. Expected message: "Group 'sysadmin' deleted."
5. Verify in terminal: `grep sysadmin /etc/group`
Expected: no output (group no longer exists)

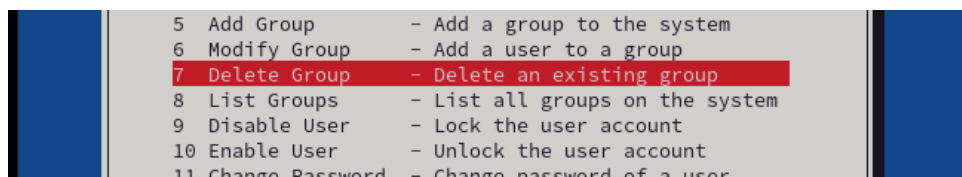


Figure 35: Choosing the option to Delete Group

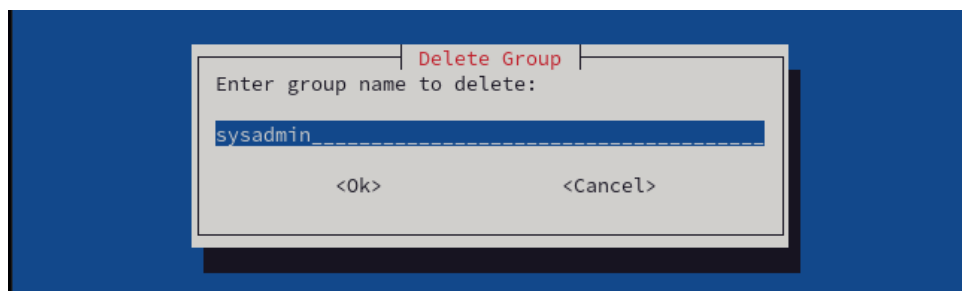


Figure 36: Delete Group – entering sysadmin

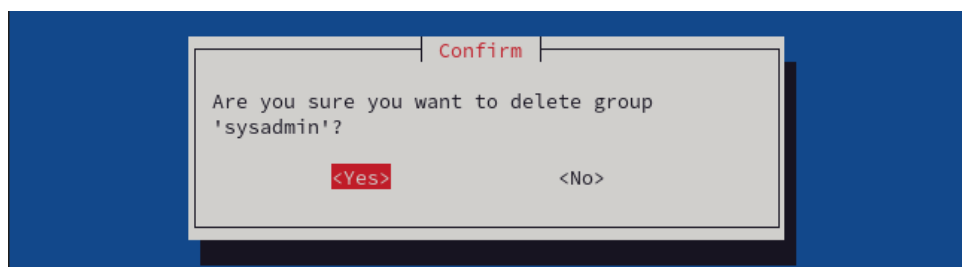


Figure 37: Delete Group – Yes/No confirmation

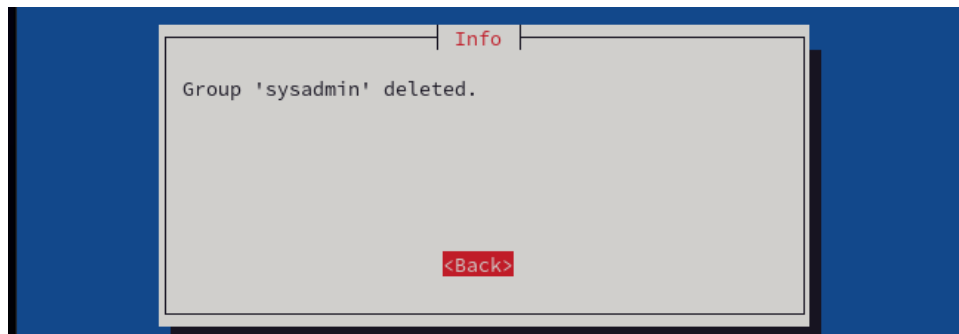


Figure 38: Success message after deletion

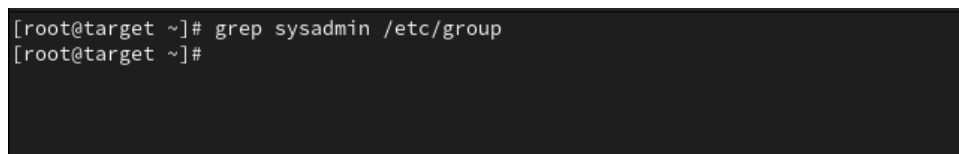


Figure 39: Terminal: grep sysadmin /etc/group – empty output

9 Command Reference Table

Menu Option	Linux Command	What It Does
Add User	<code>useradd -m</code>	Creates the user and their home directory
Modify User	<code>usermod -l</code>	Renames the login name
Delete User	<code>userdel -r</code>	Removes the user and home directory
List Users	<code>awk /etc/passwd</code>	Reads the user database file
Add Group	<code>groupadd</code>	Creates a new group
Modify Group	<code>usermod -aG</code>	Add a created user to a supplementary group
Delete Group	<code>groupdel</code>	Removes a group
List Groups	<code>/etc/group</code>	Reads the group database file
Disable User	<code>usermod -L</code>	Adds ! to password hash (locks)
Enable User	<code>usermod -U</code>	Removes ! from hash (unlocks)
Change Password	<code>chpasswd</code>	Sets password non-interactively via stdin

10

About

Ahmed Gamal

Track: System Administration

Intake 46 — ITI Alexandria

Supervised by: **Eng. Romany Nageh**

Special thanks for his guidance and professional support.

This tool was built as part of the Bash Scripting module to demonstrate practical Linux system administration using shell scripting and the `whiptail` TUI framework.